

RETO TÉCNICO • TCS

SmartBancs App

Transacciones resilientes sobre un core bancario legacy

10.000

TPS

Objetivo arquitectónico

< 2 s

Transferencia

Tiempo máximo
comprometido

Bancs

Core legacy

Se degrada con
carga directa

IA

No bloqueante

Nunca frena la
transacción

=

Consistencia

Ni perder ni
duplicar dinero

Un MVP que demostrara consistencia, desacoplamiento, concurrencia y observabilidad.

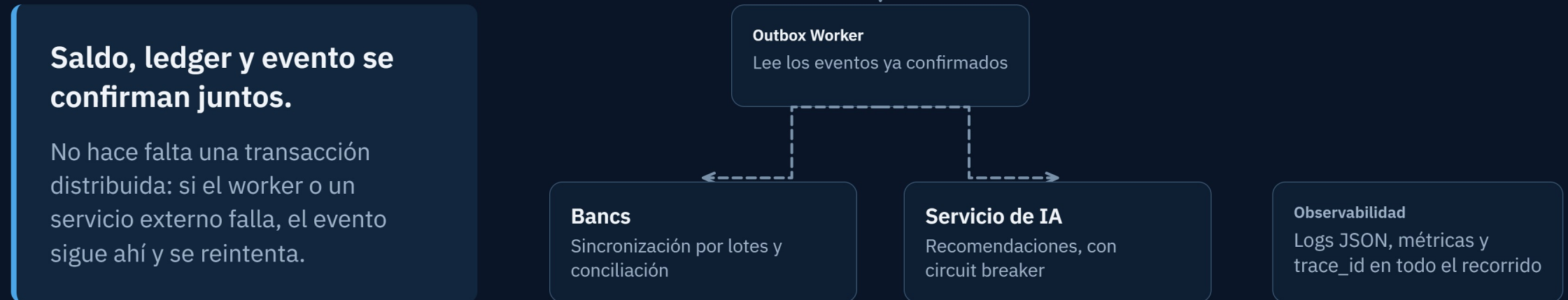
ARQUITECTURA

El cliente nunca espera por Bancs ni por la IA

CAMINO CRÍTICO



CAMINO ASÍNCRONO · DESPUÉS DEL COMMIT



Tres decisiones que protegen la transferencia

01 · CONCURRENCIA

Bloqueo pesimista en orden determinista

SELECT ... FOR UPDATE sobre las dos cuentas, siempre de menor a mayor id. El saldo se lee después de bloquear, nunca antes.

Orden invertido 29 deadlocks · orden fijo 0

30 transferencias cruzadas · prueba local

02 · PATRÓN OUTBOX

Nada externo dentro de la transacción

Saldos, ledger y evento se confirman en el mismo commit. Bancs y la IA se atienden después, desde el worker.

Sin transacción distribuida

Con la IA apagada, la transferencia completa

03 · SISTEMAS EXTERNOS

Bancs por lotes, IA con circuit breaker

A Bancs nunca se le consulta en caliente: se le envían los cambios agrupados y una conciliación compara saldos.

1.000 cambios: 37,7 s → 4,1 s

Uno por uno frente a lotes · prueba local

Del MVP hacia producción

VALIDADO EN EL MVP

- ✓ Conservación del dinero e integridad del ledger
- ✓ Idempotencia ante peticiones repetidas
- ✓ Control de concurrencia y deadlocks eliminados
- ✓ Bancos desacoplado, IA fuera del camino crítico
- ✓ Métricas, trazabilidad y diagnóstico de bloqueos

37–47 TPS

Medido · MVP local · un solo proceso

EVOLUCIÓN

10.000 TPS es el objetivo arquitectónico, no un resultado medido.

Réplicas de la API, que no guarda estado



Balanceador por delante



Pooling de conexiones



Workers escalables



Particionado de cuentas si el volumen lo exige

Consistencia primero. Escalabilidad después.

Concurrencia e idempotencia

Por qué pesimista, no optimista

En un banco muchas transferencias tocan las mismas cuentas. El optimista reintentaría, y reintentar repite trabajo justo en el pico de carga. Esperar unos milisegundos en fila cuesta menos.

Cómo desaparecen los deadlocks

Todas las transacciones piden los bloqueos en el mismo orden: menor `account_id` primero. Dos transferencias cruzadas no pueden esperarse mutuamente.

Dos transferencias simultáneas

La segunda espera al commit de la primera y lee el saldo ya actualizado. El saldo se valida después de bloquear, nunca antes.

Idempotencia

Cada petición trae una Idempotency-Key. Si llega repetida, se devuelve la respuesta original sin mover dinero. Probado con envíos simultáneos iguales.

Evidencia

29 → 0 deadlocks

30 transferencias cruzadas

Suma cero

El dinero total no cambia bajo carga

Ledger de partida doble

Cada movimiento con su contrapartida

Incidente y observabilidad

LA CADENA DEL INCIDENTE

Sube la latencia



Timeouts contra la base de datos



Pool de conexiones agotado y bloqueos



Sesión y consulta responsables, identificadas

Qué instrumenté para llegar ahí

- Latencia y errores por endpoint
- Ocupación del pool de conexiones
- Consultas bloqueadas y quién retiene el bloqueo
- Llamadas a la IA por estado: éxito y timeout
- trace_id en cada petición, de extremo a extremo

Lo que falló de verdad

Mi primer endpoint de diagnóstico consumió 2,5 GB y congeló la API durante la prueba. Lo corregí, añadí una prueba de regresión y lo documenté en el post mortem.